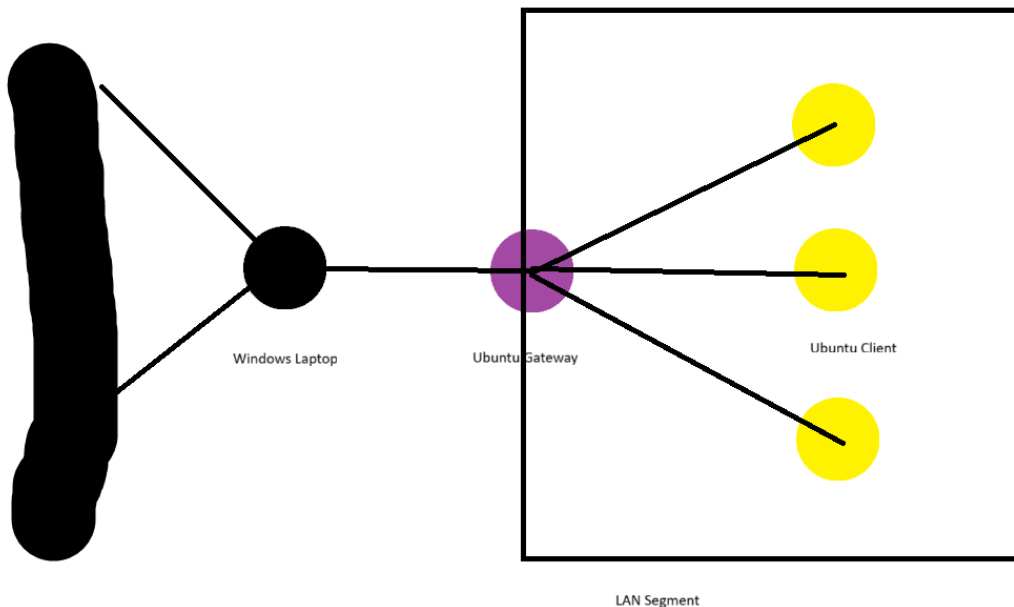


Lab4 COMSM0123 Network Security
Firewalls and Network Intrusion detection

Objectives:

In this lab, you will learn the basics and principles of network gateways, defence measures and network intrusion detection.



Please read the **instructions** before solving any questions.

1. It is advised to use two Ubuntu Server Virtual Machine, to avoid irreversible damage. One is Ubuntu Server as a **Gateway** and One is Ubuntu Server as a **Client**
2. You will need to attach **two** network interfaces to the **Gateway** VM, One NAT and one LAN Segment
3. You will be working individually on this lab.

Notes:

Create a snapshot of a virtual machine, as you will be making changes in network settings that might be irreversible.

VM Setup Summary:

- | **VM 1 - Gateway:**
- | - 2 Network Interfaces: ens160 (NAT), ens36 (LAN Segment)
- | - Runs: DHCP Server, Firewall, Zeek IDS
- | - IP: 192.168.2.1 (on LAN side)

- | **VM 2 - Client:**
- | - 1 Network Interface: (LAN Segment - same as Gateway ens36)
- | - Receives DHCP from Gateway
- | - IP: 192.168.2.x (from DHCP range)

Task 1. Gateways

Task 1.a: Linux

In Linux systems, the default gateway can be changed in “Network settings” or manually.

\$ sudo apt install -y net-tools open-vm-tools open-vm-tools-* vim

Check the current default gateway with:

```
$ ip route
OR
$ route -n
OR
ip r | grep default
```

Keep note of the result and the name of the interface:

Example: 192.168.207.2 and ens33

to change it we need to remove it:

sudo ip route delete default

Confirm deletion before proceeding.

and change it with the following:

```
$ route add default gw {IP-ADDRESS} {INTERFACE-NAME}
$ route add default gw 192.168.207.2 ens33
OR
$ ip route add 192.168.207.0/24 via 192.168.1.254
```

Check again using:

```
$ route -n
```

Question: What does this tell you about the way your computer accesses the internet?

Question: What is a default gateway?

Question: Where would you position a firewall?

Task 1.b: Linux as a Gateway

You can set up a Linux system as a network gateway.

(Note: You may need to run *sudo apt update* → *sudo apt upgrade* or install netplan first)

List the interfaces with:

```
$ ls /sys/class/net/
```

Make a backup:

```
$ sudo cp /etc/netplan/50-cloud-init.yaml /etc/netplan/50-cloud-init.yaml.bak
```

Update the netplan for the two interfaces you have.

```
$ sudo nano /etc/netplan/50-cloud-init.yaml
```

```
network:
  version: 2
  ethernets:
    ens160: # WAN – NAT interface
      addresses:
        - 192.168.1.254/24
      routes:
        - to: default
          via: 192.168.1.1 # Needs to point to NAT IP Range, same as before netplan
      nameservers:
        addresses:
          - 192.168.1.1
    ens36: # Gateway – LAN Segment
      addresses:
        - 192.168.2.1/24
      dhcp4: false
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

```
File Actions Edit View Help
GNU nano 8.1
network:
  ethernets:
    ens160:
      addresses:
        - 192.168.1.254/24
      gateway4: 192.168.1.1
      nameservers:
        addresses:
          - 192.168.1.1
    ens36:
      addresses:
        - 192.168.2.1/24
      dhcp4: false
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
  version: 2
```

Apply settings with:

```
$ sudo netplan generate
```

```
$ sudo netplan apply
```

DHCP installation:

```
$ sudo apt update
```

```
$ sudo apt install isc-dhcp-server
```

Configure DHCP:

```
$ sudo nano /etc/default/isc-dhcp-server
```

Find the line `INTERFACES="XXXXX"` and add the value for NIC2 (i.e. `ens36`).

Save and exit.

```
$ sudo nano /etc/dhcp/dhcpd.conf
```

Please remove `'option domain-name example.org;'` and append the following to the configuration file:

```
option domain-name-servers 8.8.8.8, 8.8.8.4;

option subnet-mask 255.255.255.0;
option broadcast-address 192.168.2.255;
subnet 192.168.2.0 netmask 255.255.255.0 {
range 192.168.2.20 192.168.2.250;
option routers 192.168.2.1;
}
```

```
GNU nano 8.1 /etc/dhcp/dhcpd.conf
# dhcpd.conf
#
# Sample configuration file for ISC dhcpd
#
# option definitions common to all supported networks ...
option domain-name-servers 8.8.8.8, 8.8.8.4;
option subnet-mask 255.255.255.0;
option broadcast-address 192.168.2.255;
subnet 192.168.2.0 netmask 255.255.255.0 {
range 192.168.2.20 192.168.2.250;
option routers 192.168.2.1;
}
option domain-name-servers ns1.example.org, ns2.example.org;

default-lease-time 600;
max-lease-time 7200;

# The ddns-updates-style parameter controls whether or not the server will
# attempt to do a DNS update when a lease is confirmed. We default to the
# behavior of the version 2 packages ('none', since DHCP v2 didn't
# have support for DDNS.)
ddns-update-style none;

# If this DHCP server is the official DHCP server for the local
# network, the authoritative directive should be uncommented.
#authoritative;

# Use this to send dhcp log messages to a different log file (you also
# have to hack syslog.conf to complete the redirection).
```

Restart the service:

\$ sudo service isc-dhcp-server restart

Configure IP Forwarding:

\$ sudo vim /etc/sysctl.conf

Uncomment the line: 'net.ipv4.ip_forward=1'

```
File Actions Edit View Help
#
# /etc/sysctl.conf - Configuration file for setting system variables
# See /etc/sysctl.d/ for additional system variables.
# See sysctl.conf (5) for information.
#
#kernel.domainname = example.com
# Uncomment the following to stop low-level messages on console
#kernel.printk = 3 4 1 3
#####
# Functions previously found in netbase
#
# Uncomment the next two lines to enable Spoof protection (reverse-path filter)
# Turn on Source Address Verification in all interfaces to
# prevent some spoofing attacks
#net.ipv4.conf.default.rp_filter=1
#net.ipv4.conf.all.rp_filter=1
#
# Uncomment the next line to enable TCP/IP SYN cookies
# See http://lwn.net/Articles/277146/
# Note: This may impact IPv6 TCP sessions too
#net.ipv4.tcp_syncookies=1
#
# Uncomment the next line to enable packet forwarding for IPv4
net.ipv4.ip_forward=1
#
# Uncomment the next line to enable packet forwarding for IPv6
# Enabling this option disables Stateless Address Autoconfiguration
# based on Router Advertisements for this host
#net.ipv6.conf.all.forwarding=1
-- INSERT --
```

\$ sysctl net.ipv4.ip_forward

Add a NAT forwarding rule to *iptables*:

\$ sudo iptables -t nat -A POSTROUTING -o [interface] -j MASQUERADE

([Interface] can look like eth0, enp0s8, wlan1, or even br0.)

On your **second** Ubuntu Server VM (Client):

1. Attach ONE network interface set to the same LAN Segment as the Gateway's ens36

2. Check the network configuration:

\$ ip a

3. The Client should automatically receive an IP from the DHCP server (192.168.2.20-250 range)

4. Verify you received an IP:

\$ ip a | grep 192.168.2

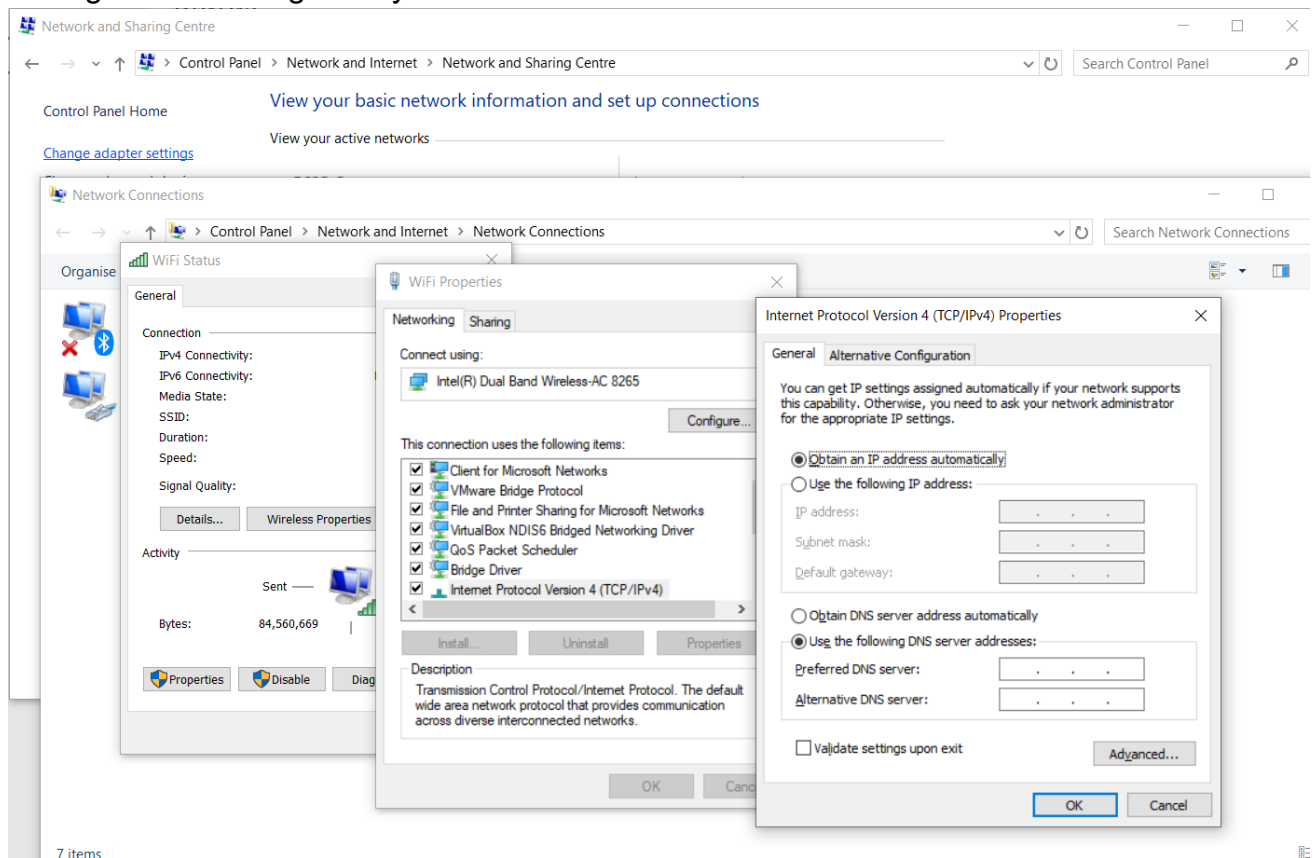
5. Test connectivity to the Gateway:
\$ ping 192.168.2.1

6. Test internet connectivity through the Gateway:
\$ ping 8.8.8.8
\$ ping google.com

Question: What happens if the Gateway's DHCP server is not running? How would you troubleshoot this?

Task 1.c: Windows

In “Network adapter settings”
Properties of “IPv4 settings” or “IP assignment”.
Change the default gateway.



For this lab, monitor **ens36** to capture traffic between the Gateway and Client VM.

In *cmd.exe* you can check the applied configuration by running *ipconfig*.

Note: A bridge can be created between two interfaces by selecting them and enabling the bridge with a right-click option.

Task 2. Firewalls

Firewalls will block or allow traffic when certain conditions are met.

Firewall rules can overlap. For example, it is possible to create a rule that blocks all traffic, unless specified otherwise by other rules.

- **INPUT** – the rules in this chain control the packets received by the server.
- **OUTPUT** – this chain controls the packets for outbound traffic.
- **FORWARD** – this set of rules controls the packets that are routed through the server.

Targets:

- **ACCEPT** - accept all traffic
- **DROP** - drop packets without response
- **REJECT** - sends an error message back to the source with “connection failure”
- **LOG** - do nothing to the packet, but log the event

If *Iptables* is not installed:

```
$ sudo apt-get install iptables iptables-persistent
```

Task 2.a: Linux - UFW

UFW (Uncomplicated Firewall) is the most user-friendly firewall in linux systems. It allows the user to manage iptables without knowing the correct syntax. You can check the status using:

```
$ sudo ufw status
```

For more information you can use:

```
$ sudo ufw status verbose
```

If it is disabled:

```
$ sudo ufw enable
```

You can block all incoming traffic by entering the following command:

```
$ sudo ufw default deny incoming
```

Or if you want to block a specific ipv4 address:

```
$ sudo ufw deny from {ip-address-here} to any
```

Other useful commands include:

```
$ sudo ufw allow OpenSSH
```

Task 2.b: IPTables

UFW is a wrapper for IPTables, to simplify the major rule manipulations. More advanced users and network administrators use IPTables directly for very precise configurations.

To list all the existing rules type:

\$ sudo iptables -L

- **-A --append** – Add a rule to a chain (at the end).
- **-C --check** – Look for a rule that matches the chain's requirements.
- **-D --delete** – Remove specified rules from a chain.
- **-F --flush** – Remove all rules.
- **-I --insert** – Add a rule to a chain at a given position.
- **-L --list** – Show all rules in a chain.
- **-N --new-chain** – Create a new chain.
- **-v --verbose** – Show more information when using a list option.
- **-X --delete-chain** – Delete the provided chain.

List all the rules in a table (without line number):

\$ sudo iptables -t nat -L -n

```
$ sudo iptables -t nat -L -n
Chain PREROUTING (policy ACCEPT)
target     prot opt source               destination

Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination

Chain POSTROUTING (policy ACCEPT)
target     prot opt source               destination
```

List all the rules in a table (with line number):

\$ sudo iptables -t filter -L -n --line-numbers

Delete rule No. 2 in the INPUT chain of the filter table:

\$ sudo iptables -t filter -D INPUT 2

Drop all the incoming packets that satisfy the <rule>:

\$ sudo iptables -t filter -A INPUT <rule> -j DROP

Accept all ICMP packets and reply to them:

\$ sudo iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT

\$ sudo iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT

\$ sudo iptables -P OUTPUT DROP ← Set default rule for OUTPUT

\$ sudo iptables -P INPUT DROP ← Set default rule for INPUT

Test the firewall rules:

From the Client VM, try to ping the Gateway:

\$ ping 192.168.2.1

If the rules are working, the ping should succeed (ACCEPT rules are in place).

Now set default DROP policies and test again:

```
$ sudo iptables -P OUTPUT DROP
```

```
$ sudo iptables -P INPUT DROP
```

From the Client VM:

```
$ ping 192.168.2.1 # This should now fail
```

Question: Why does the ping fail after setting default DROP policies?

Cleanup - Flush all chains and set to accept all

```
$ sudo iptables -F
```

```
$ sudo iptables -P OUTPUT ACCEPT
```

```
$ sudo iptables -P INPUT ACCEPT
```

Specific conditions:

```
$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

```
$ sudo iptables -A INPUT -s 192.168.0.1 -j ACCEPT
```

```
$ sudo iptables -A INPUT -m iprange --src-range 192.168.0.1-192.168.0.255 -j REJECT
```

Question: Compare the following configuration with the previous one. How would you test if it works? Which configuration is more secure?

```
$ sudo iptables -A FORWARD -p icmp --icmp-type echo-request -j DROP
```

Question: What would the following command do?

```
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

```
$ sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

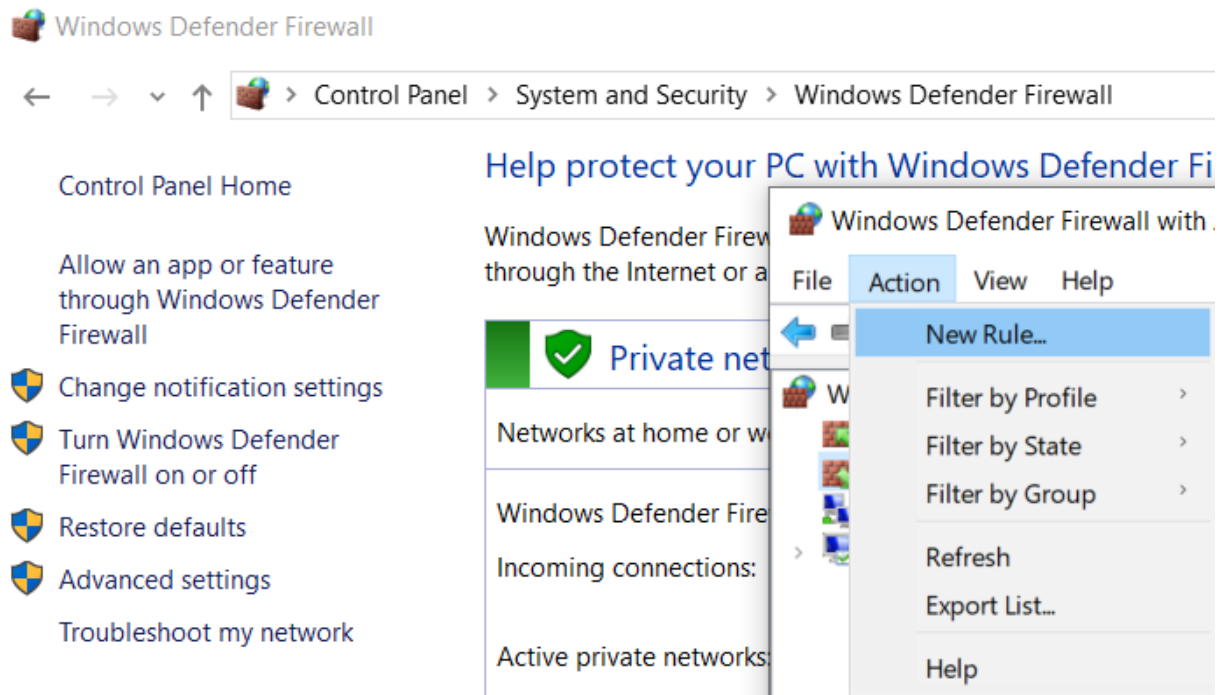
```
$ sudo iptables -A FORWARD -i eth0 -p tcp --sport 5000 -j ACCEPT
```

Task 2.c: Windows - Windows Defender Firewall

Start - Search for "Firewall" - open "Windows Firewall"

The original location is in the Control Panel - System and Security - Windows Firewall.

Enable the firewall rules to prevent windows from responding to ICMP requests and then ping the target, to check if the rules were applied correctly.



Task 3. Intrusion Detection Systems

Task 3.a: Install Zeek (On a Gateway VM)

All IDS tasks should be performed on the Gateway VM to monitor traffic passing through.

Install Zeek from the repository:

\$ sudo apt-get install zeek

Task 3.b: Check the installation

Check the installation version:

\$ zeek -V

Check the installation using the ruleset:

\$ zeek -C -r /opt/zeek/testing/btest/Traces/wikipedia.trace

This should generate several log files (conn.log, http.log, etc.) in your current directory.

Task 3.c: Configure Zeek on Ubuntu

Edit the node configuration:

\$ sudo nano /opt/zeek/etc/node.cfg

Update the interface name to match your monitoring interface (use ip a to find it):

```
[zeek]  
type=standalone  
host=localhost  
interface=ens160 # Change this to your interface name
```

Edit the networks configuration to define your local network:

```
$ sudo nano /opt/zeek/etc/networks.cfg
```

Add your local network range:

```
192.168.2.0/24 Private Network
```

Task 3.d: Test the Rulesets

Deploy the configuration:

```
$ sudo /opt/zeek/bin/zeekctl deploy
```

Check the status:

```
$ sudo /opt/zeek/bin/zeekctl status
```

View the logs:

```
$ sudo tail -f /opt/zeek/logs/current/conn.log
```

Question: Generate some network traffic (ping, browse websites) and examine the conn.log file. What information does Zeek capture about each connection?

Task 3.e: Running Zeek as a service

Start Zeek:

```
$ sudo /opt/zeek/bin/zeekctl start
```

Stop Zeek:

```
$ sudo /opt/zeek/bin/zeekctl stop
```

Restart Zeek (useful after configuration changes):

```
$ sudo /opt/zeek/bin/zeekctl restart
```

To enable Zeek to start automatically on boot:

```
$ sudo /opt/zeek/bin/zeekctl install
```

```
$ sudo crontab -e
```

Add this line if not already present:

```
*/5 * * * * /opt/zeek/bin/zeekctl cron
```

Question: What are the differences between Zeek (IDS) and a firewall? Can they work together?

Task 3.f: Generate Traffic for Analysis

On the Client VM, generate various types of traffic:

HTTP traffic:

```
$ curl http://example.com  
$ wget http://neverssl.com
```

DNS queries:

```
$ nslookup google.com  
$ dig example.com
```

Port scanning (if nmap installed):

```
$ sudo apt install nmap  
$ nmap -sT 192.168.2.1
```

On the Gateway VM, examine the Zeek logs:

```
$ cat /opt/zeek/logs/current/http.log  
$ cat /opt/zeek/logs/current/dns.log  
$ cat /opt/zeek/logs/current/conn.log  
$ cat /opt/zeek/logs/current/notice.log
```

Question: What types of suspicious activity did Zeek log from the nmap scan?

Task 4 (Optional Challenge): Convert the virtual machine into a bridge

A bridge receives traffic on one interface and outputs it on another network interface, while optionally passing it through the IDS and/or firewall.

Task 4.a: Option 1

```
$ apt-get install bridge-utils
```

Current status of the connections:

```
$ sudo nmcli con show
```

Add a new bridge:

```
$ sudo nmcli con add ifname br0 type bridge con-name br0
```

```
$ sudo nmcli con add type bridge-slave ifname enp0s31f6 master br0
```

To get IP via DHCP turn on the following:

```
$ sudo nmcli con up br0
```

Check the status of the connections:

\$ sudo nmcli con show

Task 4.b: Option 2

Create an interface:

\$ ip link add name br0 type bridge

Switch it on:

\$ ip link set dev br0 up

Connect interfaces to the bridge:

\$ ip link set dev eth1 master br0

\$ ip link set dev eth2 master br0

Link the bridge:

\$ bridge link

Show links:

\$ ip link show br0

Validate the functionality using the ping command on both IP addresses. Explain the observed behaviour.